

Secure Messenger Desktop — “Chat List + Sync Simulator”

1. Project Goal

Build a small Electron desktop app that simulates a secure messenger client, focusing on:

- Efficient local data access using SQLite
- Stable real-time sync via WebSocket
- High-performance UI for large chat lists (virtualization)
- Clean architecture and security hygiene

2. Functional Scope

Core Features

1. Chat List (Virtualized)

- Display thousands of chats efficiently
- Fields:
 - Chat ID
 - Contact Name
 - Last Message Preview
 - Timestamp
 - Unread Count

2. Message Thread View

- Paginated messages from SQLite
- Load messages in chunks (e.g., 50 at a time)
- Append new messages from WebSocket

3. Local Storage (SQLite)

- Tables:
 - chats
 - messages
- Indexed for:
 - chat_id

- timestamp
- Queries optimized for:
 - pagination
 - filtering
 - ordering

4. WebSocket Sync Simulator

- Connects to a mock server
- Receives:
 - new messages
 - updated chats
- Implements:
 - heartbeat
 - reconnect logic
 - exponential backoff
 - offline mode

5. Connection Health Logic

- Detect disconnects
- Retry with backoff
- Show status: `connected` | `reconnecting` | `offline`

6. Security Hygiene

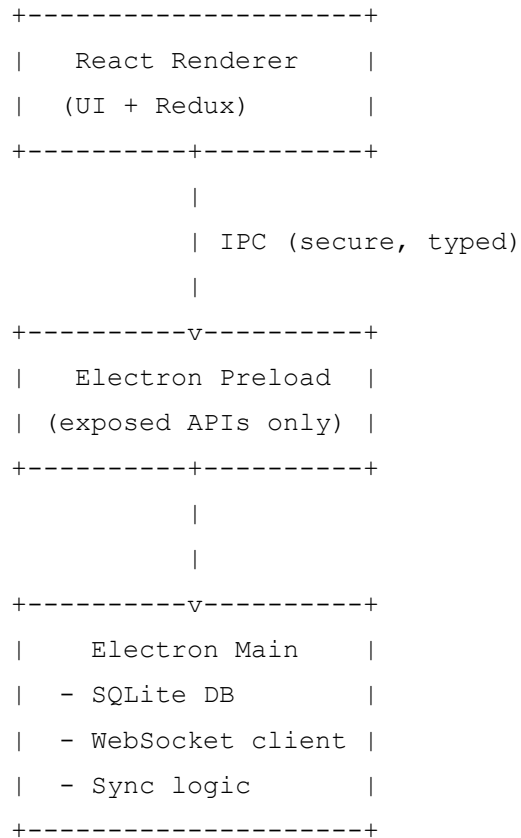
- No sensitive data in logs
- IPC boundaries enforced
- No direct DB access from React UI
- Data handled only in preload/main

3. Tech Stack

Layer	Technology
Desktop Shell	Electron
UI	React + TypeScript
State	Redux Toolkit
Local DB	SQLite (better-sqlite3 or sqlite3)

Realtime	WebSocket
Virtualization	react-window or react-virtualized

4. High-Level Architecture



5. Data Flow

Initial Load

1. App starts
 2. Main process:
 - Initializes SQLite DB
 - Seeds dummy chats + messages
 3. Renderer:
 - Requests paginated chats via IPC
 - Displays list using virtualization
-

Message Sync

1. WebSocket receives new message
2. Main process:
 - Inserts message into SQLite
 - Updates chat metadata
3. Renderer:
 - Receives IPC event
 - Updates Redux store
 - UI updates efficiently

Pagination

1. Renderer requests:
 - `getMessages(chatId, offset, limit)`
2. Main queries SQLite using:
 - indexed timestamp
3. Renderer appends results

6. Folder Structure

```
secure-messenger-desktop/  
|  
├─ electron/  
|   ├─ main.ts           # SQLite + WebSocket + IPC handlers  
|   └─ preload.ts        # Exposed APIs only  
|  
├─ src/  
|   ├─ app/  
|   |   ├─ store.ts  
|   |   └─ slices/  
|   |       ├─ chatsSlice.ts  
|   |       └─ messagesSlice.ts  
|   |  
|   └─ components/  
|       ├─ ChatList.tsx  
|       ├─ ChatItem.tsx  
|       └─ MessageList.tsx  
|   |
```

```
|   └─ services/
|     └─ ipcClient.ts
|     |
|     └─ pages/
|         └─ Home.tsx
|
|   └─ index.tsx
|
└─ server/
    └─ ws-server.ts      # Mock WebSocket server
|
└─ README.md
└─ package.json
```

Modules

Module	Responsibility
Electron Main	DB access, WebSocket sync, IPC handlers
Preload	Secure API exposure
React UI	Virtualized chat list, message thread
Redux	State management
SQLite	Local persistent storage

Data Flow

1. App loads → UI requests paginated chats
2. SQLite returns chat data
3. WebSocket receives new messages
4. Messages inserted into SQLite
5. Renderer receives IPC event
6. Redux updates UI state

Trade-Offs

- Encryption is simulated, not production-grade
- No end-to-end encryption pipeline

- WebSocket server is local mock
- No mobile client integration

Improvements With More Time

- Add real XMPP support
- Implement full E2EE boundaries
- Add message search & filtering
- Implement offline queue + resend
- Add message acknowledgments
- Add UI theming and accessibility
- Add automated tests